

Programación Concurrente y Paralela

Departamento de Computación – FCEyN – UBA
Segundo cuatrimestre de 2026

Nombre y apellido: _____

Nº orden: _____ L.U.: _____ Cant. hojas: _____

1	2	3	4	Nota

Examen simulacro – 2do. cuatrimestre de 2026

- Cada ejercicio debe realizarse en hojas separadas y numeradas. Debe identificarse cada hoja con nombre, apellido, LU y número de orden. Hojas sin identificar no serán corregidas.
- Entregue esta hoja junto al examen, la misma **no** se incluye en la cantidad total de hojas entregadas.
- **Cada código o pseudocódigo debe estar bien explicado y justificado. ¡Obligatorio!**
- Toda suposición o decisión que tome deberá justificarla adecuadamente. Si la misma no es correcta o no se condice con el enunciado no será tomada como válida y será corregida acorde.
- La devolución de los exámenes corregidos es personal. Los pedidos de revisión se realizarán por escrito, antes de retirar el examen corregido del aula.
- Los parciales tienen dos notas: I (Insuficiente) y A (Aprobado).
- Para tener la nota aprobada es necesario tener al menos dos ejercicios calificados como bien y el restante al menos regular. Además, alguno de los ejercicios calificados como bien debe ser el de monitores o el de semáforos.

Ejercicio 1.

Considere la siguiente propuesta para resolver el problema de exclusión mutua para N threads (con N fijo), donde todos los threads tienen un id distinto tal que $0 \leq \text{id} < N$:

```
1 global bool[N] esperando = [false,...,false];
2 global int cantEsperando = 0;
3 global int proximo = -1;

1 thread (id) {
2     //Seccion No Critica
3     bool soyprimero = anotarse(id);
4     if (soyprimero) llamarProximo();
5     while (proximo!=id);
6     //Seccion Critica
7     bool soyultimo = desanotarse(id);
8     if (!soyultimo) llamarProximo();
9     //Seccion No Critica
10 }
```

Dadas las siguientes funciones.

```
1 bool anotarse(int idThread) {
2     esperando[idThread]=true;
3     cantEsperando++;
4     return (cantEsperando==1);
5 }

1 bool desanotarse(int idThread) {
2     esperando[idThread]=false;
3     cantEsperando--;
4     return (cantEsperando==0);
5 }

1 void llamarProximo() {
2     int res = -1;
3     for i in {N-1,...,0} {
4         if (esperando[i]) res = i;
5     }
6     proximo = res;
7 }
```

Responder a las siguientes preguntas:

- Si se considera que `anotarse()`, `desanotarse()` y `llamarProximo()` no son atómicas, muestre adecuadamente que esta propuesta *no* resuelve el problema de la exclusión mutua.
- Considerando que `anotarse()`, `desanotarse()` y `llamarProximo()` son atómicas, ¿Resuelve esta propuesta el problema de la exclusión mutua? Justifique apropiadamente.
- ¿Alcanza con que `anotarse()`, `desanotarse()` y `llamarProximo()` sean atómicas para que el análisis del inciso b) siga valiendo al ejecutar este algoritmo en una arquitectura real, donde corre Java?

Para cada propiedad que en el inciso b) haya dado por válida, indique si el algoritmo se mantendría igual o si haría falta agregar restricciones para sostener esa garantía, y en ese caso cuál sería la restricción mínima que agregaría a nivel lenguaje y a nivel arquitectónico.

Para cada propiedad que en el inciso b) haya dado por no válida, indique si en este nuevo escenario sigue fallando por el mismo motivo, si aparecen motivos adicionales para que falle, o si las garantías de la arquitectura alcanzarían para cambiar esa conclusión.

Solución.

- Si llegan dos threads y pierden el incremento de `cantEsperando`, se puede lograr que `anotarse()` devuelva `true` para dos threads, logrando dos invocaciones seguidas a `llamarProximo()`. A partir de ahí, se puede perder la propiedad de Mutex y la de Garantía de Entrada (y, potencialmente, Deadlock):
 - **GdE:** Se hacen los dos llamados a `llamarProximo()` uno detrás de otro, dejando entrar al thread de `id` más bajo a la SC. Al salir, `desanotarse(id)` decrementa `cantEsperando`, dejándolo en 0 y por lo tanto no volviendo a llamar a `llamarProximo()`. El thread de `id` más alto se quedará esperando por siempre.
 - **Aus. Deadlock:** Esta misma estrategia, aplicada con 3 threads, puede derivar en una situación donde *dos* threads se quedan esperando para entrar en la sección crítica (Llegan los tres juntos, se pierden dos sumas sobre `cantEsperando`, luego se ejecutan tres `llamarProximo`, que dejan como `proximo` al thread de `id` más bajo, este entra, al salir no hace `llamarProximo()` y los otros quedan bloqueados. La definición de si es una espera circular es debatible, pero es un potencial escenario de Deadlock).
 - **Mutex:** Nuevamente con 3 threads, nos aseguramos de que los dos threads con `id` más alto sean los que tengan una posible invocación a `llamarProximo()`. De esos dos, el que tenga `id` más bajo entra en la SC. En este momento se deja al thread con `id` más bajo de los tres setear en `true` su flag, se ejecuta la llamada adicional a `llamarProximo()`. Esto hará que el thread con `id` bajo entre en la S.C. mientras que ún hay otro thread en ella.

Alternativamente, basta con que dos threads hagan el incremento de `cantEsperando` concurrentemente, de manera que su valor quede en 2 y por lo tanto ambos llamados a `anotarse` devuelvan `false` para estar ante otro escenario de Deadlock/pérdida de Garantía de Entrada.

- Todos los threads tienen un `id` único, por lo que para que dos threads puedan entrar en la sección crítica de manera concurrente es necesario que haya una invocación a `llamarProximo()` (la única manera en que `proximo` cambie de valor) mientras hay un thread en la sección crítica. Esto no es posible: Debido a que `anotarse()` es atómica, todo thread que llegue necesariamente verá un valor de `cantEsperando` de al menos 1 y por lo tanto no provocará un llamado a `llamarProximo()`. La GdE se basa en la consistencia de `cantEsperando`. Siendo `anotarse()` atómica, cada thread que llega garantiza un incremento sobre `cantEsperando`. Un único thread invocará `llamarProximo()` antes de su sección crítica porque la comparación con 1 sucede en el contexto atómico. Eso habilitará al thread de `id` más bajo a entrar, y en tanto no sea el último, este habilitará al siguiente a ejecutar. Notar que es clave que todo thread quiere acceder a la sección crítica **una única vez**, de no ser el caso podría haber un escenario de starvation con tres threads, donde el que tiene `id` más alto siempre es desplazado.
- El problema es la lectura de los valores de `cantEsperando` y `proximo`. Suponiendo que no hay barreras adicionales y que las variables no están declaradas como `volatile`, puede leer valores viejos de las mismas produciendo problemas similares a los del inciso a). Luego, perdemos mutex en el momento en que dos threads leen el `cantEsperando` que hizo otro thread en 0 (aunque ya lo haya incrementado), provocando potencialmente la misma situación de pérdida de Mutex que mencionamos anteriormente. A priori, la situación de pérdida de GdE/Ausencia de Deadlocks no se termina dando porque finalmente cada thread está ejecutando cómo si fuera el único.

Ejercicio 2.

Se desea modelar el procesamiento de transferencias en un banco con N cuentas. Cada transferencia se ejecuta mediante una operación `transferir(origen, destino, monto)`, con `monto` positivo, que mueve dinero de la cuenta `origen` a la cuenta `destino` (una transferencia nunca tiene la misma cuenta como origen y destino). El banco deja que las cuentas queden en rojo, pero solo hasta cierto punto: una transferencia se realiza únicamente si la cuenta de origen no queda con un saldo menor que $-L$, donde L es una constante conocida e igual para todas las cuentas. Si no es así, la transferencia se descarta.

Mientras una transferencia se está procesando, ningún otro proceso del banco puede ver ni modificar el saldo de ninguna de las dos cuentas involucradas. Los pedidos de transferencia llegan continuamente y de forma independiente (modele cada uno como un thread), sin ningún control centralizado sobre cuántos llegan a la vez, entre qué cuentas, ni en qué orden. Dos transferencias que no compartan ninguna cuenta deben poder procesarse simultáneamente.

Además, cada tanto el banco necesita saber cuántas cuentas quedaron en rojo, es decir con saldo negativo, y para eso cuenta con una operación `cuentasEnRojo()`. El resultado tiene que corresponder a un estado real del banco, sin transferencias a medio procesar. Como es un informe interno y nadie lo está esperando, se lo corre de madrugada, que es cuando el movimiento afloja: transferencias siguen llegando, pero hay ratos en los que no queda ninguna dando vueltas. Al banco no le molesta que el informe tenga que esperar su momento para arrancar. Lo que no tolera es lo contrario: que un informe que todavía no empezó a contar le haga esperar a una transferencia.

- Proponga, en pseudocódigo, una solución usando semáforos que resuelva el problema. Puede usar el constructor `Semaphore(n)` y las primitivas `acquire()` y `release()`. Salvo que se indique lo contrario, los semáforos se asumen débiles. La solución no debe tener deadlocks ni race conditions. Justifique todas las decisiones tomadas.
- Al banco le interesa que cada transferencia se procese lo antes posible, y le molesta que un pedido quede esperando mientras otros que llegaron después pasan adelante. ¿Puede ocurrir eso en su solución? ¿Hay algún límite para la cantidad de veces que un mismo pedido puede ser sobrepasado? Justifique su respuesta y, si corresponde, describa una traza.
- Suponga ahora que una transferencia puede mover dinero de una cuenta a varias a la vez, mediante una operación `transferirMultiple(origen, destinos, montos)`, donde todas las cuentas involucradas son distintas y la transferencia se descarta si el origen no alcanza para el total. Explique cómo modificaría su solución y si su justificación de ausencia de deadlock sigue siendo válida. No se pide reescribir la solución completa.

Solución.

- Una posible solución:

```
1 balance = [0] * N
2 accountMutex = [Semaforo(1)] * N
3
4 int activeTransfers = 0
5 Semaphore counterMutex = Semaphore(1)
6 Semaphore noTransfers = Semaphore(1)
7
8 void transferir(int origen, int destino, int monto) {
9     counterMutex.acquire()
10    activeTransfers++
11    if (activeTransfers == 1) {
12        noTransfers.acquire()
13    }
14    counterMutex.release()
15
16    int first = min(origen, destino)
17    int second = max(origen, destino)
18    accountMutex[first].acquire()
19    accountMutex[second].acquire()
```

```

20     if (balance[origen] - monto >= -L) {
21         balance[origen] -= monto
22         balance[destino] += monto
23     }
24     accountMutex[second].release()
25     accountMutex[first].release()
26
27     counterMutex.acquire()
28     activeTransfers--
29     if (activeTransfers == 0) {
30         noTransfers.release()
31     }
32     counterMutex.release()
33 }
34
35 int cuentasEnRojo() {
36     noTransfers.acquire()
37     int count = 0
38     for (int i = 0; i < N; i++) {
39         if (balance[i] < 0) {
40             count++
41         }
42     }
43     noTransfers.release()
44     return count
45 }

```

La solución tiene dos partes independientes: el acceso a las cuentas de una transferencia, y la convivencia entre las transferencias y el informe.

Un semáforo por cuenta. Cada cuenta tiene su propio `accountMutex[i]`, y una transferencia toma los de sus dos cuentas antes de leer o escribir saldos. Así nadie más puede ver ni modificar esas cuentas mientras la transferencia está en curso, y dos transferencias sin cuentas en común toman semáforos distintos y avanzan a la vez. Un único semáforo para todo el banco también daría exclusión, pero serializaría todas las transferencias, que es justo lo que el enunciado prohíbe.

Si se consultara el saldo antes de tomar los semáforos, dos transferencias desde la misma cuenta podrían aprobarse mirando el mismo saldo. Por ejemplo, con $L = 100$ y la cuenta 0 en 0, dos transferencias de 80 desde la cuenta 0 leen ambas $0 - 80 \geq -100$, ambas debitan y la cuenta termina en -160 . Con el chequeo y la modificación dentro de la misma sección crítica, la segunda ve el saldo ya debitado (-80) y se descarta.

Si cada transferencia tomara primero el semáforo de `origen` y después el de `destino`, hay deadlock. Basta con `transferir(0, 1, m)` y `transferir(1, 0, m)` concurrentes: la primera toma `accountMutex[0]`, la segunda toma `accountMutex[1]`, y cada una queda esperando el semáforo que tiene la otra. Lo mismo pasa con cualquier ciclo de k transferencias $i \rightarrow (i + 1) \bmod k$. Por eso las dos cuentas se toman siempre de menor a mayor número, sin importar cuál es el origen y cuál el destino.

Ausencia de deadlock El código cumple la siguiente propiedad: *si una transferencia está bloqueada esperando `accountMutex[a]`, todos los semáforos de cuenta que tiene tomados son de cuentas menores que a* , porque los pide en orden creciente. Supongamos que hay un deadlock, o sea un ciclo de transferencias $T_1, \dots, T_m$ donde cada T_i espera `accountMutex[a_i]` y ese semáforo lo tiene tomado T_{i+1} (con $T_{m+1} = T_1$). Como T_{i+1} tiene tomado a_i y está esperando a_{i+1} , por la propiedad vale $a_i < a_{i+1}$. Recorriendo el ciclo queda $a_1 < a_2 < \dots < a_m < a_1$, lo cual es absurdo. Luego no puede haber un ciclo de espera entre transferencias.

Recorrer las cuentas de a una mientras las transferencias siguen corriendo no da un estado real. Por ejemplo, con la cuenta 0 en -10 y la cuenta 1 en 100 (una cuenta en rojo), el informe lee la cuenta 0 y la cuenta en rojo. Justo después se ejecuta `transferir(1, 0, 120)`, que deja 110 y -20 . El informe lee la cuenta 1 y también la cuenta en rojo. Devuelve 2, cuando en todo momento hubo exactamente una cuenta en rojo.

Por eso el informe necesita contar en un momento en que no haya ninguna transferencia a medio

procesar. Para eso está `noTransfers`: la primera transferencia que entra cuando no había ninguna lo toma, y la última que sale lo libera. Las transferencias no se bloquean entre sí por este semáforo (solo lo toca la que encuentra el contador en 0), así que siguen pudiendo correr en paralelo. El contador `activeTransfers` lo leen y modifican varios threads, por eso se protege con `counterMutex`. Notar que `noTransfers` lo puede liberar una transferencia distinta de la que lo tomó, algo que un semáforo permite y un lock no.

Cuando `cuentasEnRojo` logra tomar `noTransfers`, no hay transferencias en curso y ninguna puede empezar hasta que lo libere. Entonces puede leer los N saldos sin tomar los semáforos de cuenta, y lo que lee es un estado real del banco. Si llega una transferencia mientras el informe cuenta, es la que encuentra el contador en 0, queda esperando `noTransfers` mientras sostiene `counterMutex`, y las demás esperan en `counterMutex`. Esa espera es aceptable porque el informe ya empezó a contar, y dura lo que tarda en recorrer N saldos.

El requerimiento de que el informe no demore transferencias. Un informe que todavía no empezó a contar está bloqueado en `noTransfers.acquire()`, y eso no afecta a ninguna transferencia: las que llegan encuentran el contador en al menos 1 y pasan de largo. La contracara es que el informe puede esperar mucho, incluso indefinidamente si siempre hay alguna transferencia en curso. El enunciado acepta esto y además asegura que de madrugada hay momentos sin transferencias, en los que el contador llega a 0 y el informe puede arrancar.

Esto descarta otras soluciones que también dan un estado consistente. Por ejemplo, que el informe tome los N semáforos de cuenta en orden, cuente y los libere. No hay deadlock (respeto el mismo orden), pero mientras va tomando semáforos todavía no empezó a contar y ya está frenando a las transferencias que necesitan las cuentas que tiene tomadas. Lo mismo pasa si se agrega un molinete para que el informe bloquee a las transferencias nuevas mientras espera que terminen las que están en curso.

Ausencia de deadlock en general. Además de las cuentas, los únicos otros puntos de espera son `counterMutex` y `noTransfers`. Quien tiene `counterMutex` a lo sumo espera `noTransfers`, y solo si el contador estaba en 0, o sea si `noTransfers` lo tiene un informe. Ese informe no espera nada más, termina de contar y lo libera. Por otro lado, las transferencias hacen la entrada y la salida del contador sin tener tomado ningún semáforo de cuenta, así que no se mezclan con el ciclo analizado arriba.

Ejercicio 3.

Inspirada en las noches de juegos de la facultad, Victoria decidió organizar algo similar en su casa para sus múltiples grupos de amigos, que en el fondo se detestan entre sí pero ninguno se lo pierde. Para eso cuenta con M mesas, donde la mesa i tiene `lugares[i]` asientos. A lo largo de la noche llegan grupos de amigos de distintos tamaños para jugar. Un grupo de k amigos necesita una mesa con al menos k asientos. El grupo no se separa y ocupa la mesa por completo aunque le sobren lugares, solo para no tener que compartirla con gente de otro grupo. Si un grupo al llegar no encuentra ninguna mesa libre, tienen que esperar hasta que se desocupe una, mirando con bastante rencor a los que la están ocupando.

Como Victoria sabe de estos problemas, nos pidió que simuláramos una posible solución. Lo único que nos pide es que, tarde o temprano, todos sus grupos de amigos consigan sentarse a jugar.

Modele esto mediante dos operaciones, `sentarse(k)` y `levantarse(mesa)`. Cada grupo se modela con un thread que, al llegar, invoca `sentarse(k)`, que le indica la mesa que le tocó y no retorna hasta conseguirla. Luego el grupo juega (esto no hace falta modelarlo) y, cuando termina, invoca `levantarse(mesa)` para liberarla.

Puede asumir que para todo grupo existe al menos una mesa en la que entra, y que todo grupo que se sienta se va en algún momento.

- Resuelva, en pseudocódigo, usando monitores con semántica de Mesa. Las colas de condición no son fair y no hay spurious wakeups. La solución no debe tener deadlocks ni race conditions. Justifique todas las decisiones tomadas.
- Explique en palabras cómo cambiaría su solución si el monitor tuviera semántica de Hoare. No se pide el código.

Solución.

a) Una posible solución:

```

1  monitor Mesas {
2      bool[] libre = [true] * N
3      int proximoNumero = 0
4      int atendiendo = 0
5      Condition cambio
6
7      int sentarse(int k) {
8          int miNumero = proximoNumero
9          proximoNumero++
10         while (miNumero != atendiendo or mesaQueSirve(k) == -1) {
11             cambio.wait()
12         }
13         int mesa = mesaQueSirve(k)
14         libre[mesa] = false
15         atendiendo++
16         cambio.signalAll()
17         return mesa
18     }
19
20     void levantarse(int mesa) {
21         libre[mesa] = true
22         cambio.signalAll()
23     }
24
25     // devuelve una mesa libre donde entren k, o -1 si no hay
26     int mesaQueSirve(int k) {
27         for (int i = 0; i < M; i++) {
28             if (libre[i] and lugares[i] >= k) {
29                 return i
30             }
31         }
32         return -1
33     }
34 }

```

La solución más directa sería que **sentarse** espere con **while (mesaQueSirve(k) == -1) cambio.wait()** y que **levantarse** haga **signalAll**. No tiene deadlock ni race conditions, pero no garantiza que todos se sienten. Con dos mesas, **lugares = {2, 6}**, supongamos que un grupo de 6 espera y la mesa grande está ocupada por un grupo de 2 (la chica también está ocupada). Cuando el grupo de 2 se levanta, el de 6 se despierta, pero con semántica de Mesa no corre enseguida: tiene que volver a entrar al monitor. Antes de que lo haga entra un grupo de 2 recién llegado, que no encuentra libre la mesa chica y ocupa la grande. El grupo de 6 vuelve a chequear, no hay mesa y vuelve a dormir. Si esto se repite cada vez, el grupo de 6 no se sienta nunca. Elegir siempre la mesa más chica en la que entra el grupo no lo arregla, porque en la traza la chica está ocupada.

El problema es que un grupo que llegó después le gana la mesa a uno que está esperando. Para evitarlo, los grupos se sientan en orden de llegada, con números: cada grupo saca un número (**miNumero**) al entrar, y solo puede sentarse el que tiene el número que se está atendiendo (**atendiendo**), cuando además hay una mesa libre en la que entra. Al sentarse avanza **atendiendo** y le toca al siguiente.

Race conditions. Todo el estado compartido (**libre**, **proximoNumero** y **atendiendo**) está dentro del monitor, así que se accede con exclusión mutua. En particular, tomar el número y avanzarlo ocurre de forma atómica, y la mesa se busca y se marca ocupada dentro de la misma llamada, así que dos grupos no pueden quedarse con la misma mesa.

Por qué while y no if. Con semántica de Mesa, entre que un grupo es despertado y vuelve a ejecutar dentro del monitor pueden entrar otros threads y cambiar el estado. Además se despierta a todos, y a la mayoría no les toca el turno. Por eso cada grupo vuelve a evaluar su condición al despertar.

Por qué `signalAll` y no `signal`. Todos los grupos esperan en la misma condición, pero en cada momento solo uno puede avanzar, el que tiene el turno. Como la cola de condición no es fair, un `signal` podría despertar a cualquier otro grupo, que vuelve a dormir, y el aviso se pierde. Por ejemplo, el grupo con el turno espera una mesa, se libera una en la que entra, el `signal` de `levantarse` despierta a otro grupo, y el que tenía el turno sigue dormido con una mesa disponible. Si después se levantan todos los que estaban sentados, todos terminan esperando y nadie los despierta. Con `signalAll` el grupo con el turno siempre se entera.

Por la misma razón `sentarse` también hace `signalAll` después de avanzar `atendiendo`: el siguiente grupo puede estar esperando y tener ya una mesa libre donde entra, y tiene que despertarse para verlo. En general, cada vez que se modifica algo de lo que depende la condición del `while` (`libre` o `atendiendo`) se despierta a todos.

Veamos que el grupo con número t se sienta, suponiendo que todos los de número menor ya lo hicieron. Cuando `atendiendo` llega a t , el grupo t es el único que puede pasar la condición del `while`, así que ningún otro grupo ocupa mesas mientras tanto. Por hipótesis existe una mesa en la que entra. Si está libre, el grupo la toma. Si no, la ocupa un grupo que se va en algún momento, y como nadie más puede sentarse, al levantarse esa mesa queda libre y se hace `signalAll`. En ambos casos el grupo t recibe un aviso después del último cambio relevante (el que puso `atendiendo` en t o el que liberó la mesa), vuelve a evaluar la condición y la encuentra verdadera. Como no hay spurious wakeups ni otro grupo que pueda tomar esa mesa, se sienta. Esto supone que todo thread que quiere entrar al monitor eventualmente entra.

Ausencia de deadlock. El único lugar donde se espera es `cambio.wait()`, y el argumento anterior muestra que el grupo con el turno siempre termina sentándose. Los grupos sentados no esperan nada. No puede quedar todo el mundo bloqueado mientras haya grupos esperando.

Esta solución es bastante conservadora: si el grupo con el turno espera una mesa grande, los que vienen detrás esperan aunque haya una mesa chica libre en la que entrarían. Así que no es del todo eficiente. El enunciado solo pide que todos se sienten tarde o temprano, y atender en orden de llegada lo garantiza de la forma más simple.

Una solución mas eficiente. Se puede mantener la garantía de que todos se sientan sin frenar a toda la fila detrás del primero. La observación es que en la traza del principio el problema no es que un grupo se siente fuera de orden, sino que se quede con una mesa que le servía a alguien que estaba esperando desde antes. Entonces la regla pasa a ser: *un grupo puede ocupar una mesa libre en la que entra siempre que ningún grupo que llegó antes que él y sigue esperando entre en esa mesa*. Cada mesa queda, en la práctica, reservada para el grupo más antiguo que la puede usar.

```

1  class Pedido {
2      int cantidad
3      Condition puedeSentarse
4
5      Pedido(int k) {
6          cantidad = k
7      }
8  }
9
10 monitor Mesas {
11     bool[] libre = [true] * N
12     List<Pedido> cola = []
13
14     int sentarse(int k) {
15         Pedido pedido = Pedido(k)
16         cola.add(pedido)
17         while (mesaDisponible(pedido) == -1) {
18             pedido.puedeSentarse.wait()
19         }
20         int mesa = mesaDisponible(pedido)
21         libre[mesa] = false
22         cola.remove(pedido)
23         for (int i = 0; i < M; i++) {
24             if (libre[i]) {

```

```

25         despertarPrimeroQueEntra(i)
26     }
27 }
28 return mesa
29 }
30
31 void levantarse(int mesa) {
32     libre[mesa] = true
33     despertarPrimeroQueEntra(mesa)
34 }
35
36 // despierta al primer grupo de la cola que entra en la mesa i
37 void despertarPrimeroQueEntra(int i) {
38     for (Pedido otro : cola) {
39         if (otro.cantidad <= lugares[i]) {
40             otro.puedeSentarse.signal()
41             return
42         }
43     }
44 }
45
46 // una mesa libre donde entra el pedido y que no le sirve
47 // a nadie que llego antes, o -1 si no hay
48 int mesaDisponible(Pedido pedido) {
49     for (int i = 0; i < M; i++) {
50         if (libre[i] and lugares[i] >= pedido.cantidad
51             and not reservadaAntes(pedido, i)) {
52             return i
53         }
54     }
55     return -1
56 }
57
58 // true si algun grupo anterior al pedido en la cola entra en la mesa i
59 boolean reservadaAntes(Pedido pedido, int i) {
60     for (Pedido otro : cola) {
61         if (otro == pedido) {
62             return false
63         }
64         if (otro.cantidad <= lugares[i]) {
65             return true
66         }
67     }
68     return false
69 }
70 }

```

Pedido representa el pedido de un grupo: guarda su tamaño (**cantidad**) y una variable de condición propia (**puedeSentarse**) donde ese grupo, y solo ese, espera. Como toda variable de condición, pertenece al monitor, aunque la guardemos en el pedido. Se usa un objeto nuevo por llamada, y no el número *k*, porque puede haber varios grupos del mismo tamaño esperando y cada uno tiene que poder encontrar su posición en la cola, sacarse de ella y ser despertado individualmente. La cola mantiene el orden de llegada, así que “llegó antes” es “está antes en la cola”.

En la traza del principio, con **lugares** = {2, 6}, cuando se libera la mesa grande el grupo de 6 está en la cola y entra en ella, así que el grupo de 2 que llega después no puede ocuparla, aunque la vea libre. En cambio, si la que está libre es la mesa chica, el grupo de 2 se sienta ahí sin esperar al de 6, porque el de 6 no entra en esa mesa. La solución anterior lo hacía esperar.

Por qué una condición por pedido. Si todos esperaran en una misma condición, habría que usar

signalAll, porque la cola de condición no es fair y un **signal** podría despertar a un grupo que no puede sentarse, perdiendo el aviso. Eso despierta a toda la cola cada vez que se libera una mesa, y casi todos vuelven a dormir después de recorrer la cola entera para evaluar su condición. Con una condición por pedido se sabe exactamente a quién despertar: cuando una mesa j está libre, el único que puede ocuparla es el primer grupo de la cola que entra en ella (los anteriores no entran, y los posteriores tienen a ese grupo adelante). Eso es lo que hace **despertarPrimeroQueEntra(j)**. Como en cada condición espera a lo sumo un thread, que la cola de condición no sea fair deja de importar.

La condición de un grupo depende de **libre** y de los grupos que tiene adelante en la cola. Hay cuatro cambios posibles:

- **levantarse** libera la mesa j : el primero de la cola que entra en j pasa a poder sentarse, y se lo despierta a él solo.
- Un grupo se sienta y sale de la cola: deja de reservar las mesas libres en las que entraba, y para esas mesas el primero que entra puede pasar a ser otro grupo, que estaba detrás. Por eso, al salir, **sentarse** llama a **despertarPrimeroQueEntra** para cada mesa libre.
- Un grupo ocupa una mesa: eso no habilita a nadie.
- Un grupo llega y se agrega al final de la cola: solo cambia la condición de los que llegan después de él, que todavía no existen. Si hay una mesa libre que le sirve, lo ve él mismo al evaluar su condición, sin haber soltado el monitor, así que no hace falta avisar a nadie.

Al salir de la cola se llama a **despertarPrimeroQueEntra** para todas las mesas libres, no solo para las que reservaba el grupo que se sentó. Es más simple y no despierta a nadie de más: para una mesa libre cuyo primer interesado no cambió, ese grupo ya fue despertado cuando la mesa quedó disponible para él, así que no está esperando en su condición y el **signal** no tiene efecto.

Race conditions. Igual que antes, todo el estado (**libre** y **cola**) vive dentro del monitor. Buscar la mesa, marcarla ocupada y salir de la cola ocurre en la misma llamada, sin soltar el monitor en el medio. Tampoco se pierde un aviso por llegar antes del **wait**: un grupo se agrega a la cola y evalúa su condición en la misma llamada, así que nadie puede intentar despertarlo antes de que esté esperando.

Sea G un grupo en la cola y supongamos que todos los que llegaron antes eventualmente se sientan. Como cada grupo se sienta una sola vez, a partir de cierto momento G es el primero de la cola. Sea j una mesa en la que entra G , que existe por hipótesis. Desde ese momento ningún otro grupo puede ocupar j : todos los demás de la cola llegaron después de G , y G entra en j , así que **reservadaAntes** da verdadero para ellos. Si j está ocupada, el grupo que la tiene se va en algún momento, y **levantarse** despierta al primero de la cola que entra en j , que es G . Si j ya estaba libre cuando G quedó primero, lo despertó el grupo que salió de la cola, que recorre las mesas libres. En ambos casos G vuelve a evaluar su condición, encuentra al menos j disponible y se sienta (en j o en otra mesa que también pueda usar).

Ausencia de deadlock. Los únicos puntos de espera son los **puedeSentarse.wait()**, y el argumento anterior muestra que el primero de la cola siempre termina sentándose.

Con esta regla ninguna mesa queda libre mientras haya esperando un grupo que entra en ella, salvo el tiempo que tarda el grupo despertado en volver a entrar al monitor. Y cada cambio despierta solo a quien puede aprovecharlo: **levantarse** despierta a lo sumo a un grupo, y la salida de la cola a lo sumo a uno por mesa libre. El precio es una solución bastante más difícil de escribir y de justificar que la de la cola en orden.

- b) Con semántica de Hoare, el thread despertado por un **signal** corre inmediatamente, con el estado exactamente como lo dejó quien hizo el **signal**. Si solo se despierta a un grupo cuando su condición es verdadera, esa condición se sigue cumpliendo cuando corre, y el **while** se puede cambiar por un **if**.

Pero eso no alcanza con cambiar los **while** por **if** en esta solución. Con **signalAll** se despierta a grupos cuya condición es falsa (los que no tienen el turno), y si pasaran por un **if** se sentarían fuera de turno. Y un **signal** sobre la condición compartida puede despertar a cualquiera, porque la cola no es fair. Por eso hay que despertar exactamente al grupo con el turno, y solo cuando le sirve una mesa.

Si el monitor ofrece **signalAll**, otra opción es dejar la solución de Mesa como está: con **while** sigue siendo correcta con semántica de Hoare, porque volver a chequear la condición nunca está de más. Lo que se gana con Hoare es poder despertar a un solo grupo y no volver a chequear, a cambio de tener que calcular bien a quién despertar.

Ejercicio 4.

Se requiere diseñar un contador concurrente optimizado para alta escritura y baja lectura. Al crearse, la estructura inicializa un arreglo de N variables enteras para distribuir el nivel de concurrencia. Las operaciones de incremento se distribuirán entre estas variables para minimizar la contención. Por otro lado, la operación de lectura consolidará el total sumando el valor de las N variables. A continuación se muestra una implementación secuencial.

```
1  class Contador {
2
3      private int[] valor = new int[n];
4      private int tamano;
5
6      Contador(int n) {
7          tamano = n;
8          for (int i = 0; i < n; i++) {
9              valor[i] = 0;
10         }
11     }
12
13     public void inc(int id) {
14         valor[id % tamano]++;
15     }
16
17     public int get() {
18         int total = 0;
19         for (int i = 0; i < tamano; i++) {
20             total = total + valor[i];
21         }
22         return total;
23     }
24 }
25 }
```

- a) Dar una solución de granularidad fina que permita la invocación concurrente de un incremento y de obtener el valor actual. Justificar la correctitud de su solución (en términos de linearizabilidad).
- b) Analizar el progreso del programa (presencia/ausencia de deadlocks, presencia/ausencia de starvation).
- c) Proponer una solución alternativa utilizando el enfoque de concurrencia *lock-free* (basada en operaciones CAS) para la operación de incremento. (**Nota:** Para este punto, no es necesario preocuparse por la implementación de `get()`).
- d) Considere que la operación `get()` se redefine mediante el siguiente algoritmo

```
1  public int get() {
2      while (true) {
3          int total1 = 0;
4
5          for (int i = 0; i < tamano; i++) {
6              total1 = total1 + valor[i];
7          }
8          int total2 = 0;
9
10         for (int i = 0; i < tamano; i++) {
11             total2 = total2 + valor[i];
12         }
13
14         if (total1 == total2) {
15             return total1;
16         }
17     }
18 }
```

¿Es esta solución correcta (linearizable)? Justifique.

e) Indicar si en su solución el método `get()` es *wait-free*. Justificar.

Inciso extra (por fuera del parcial):

- Analice la correctitud (linearizabilidad) de una solución híbrida donde la operación `get()` mantiene la estrategia de granularidad fina definida en el Punto 1 (utilizando locks), pero las otras operaciones son las que utilizan CAS, con los incrementos optimistas.

Solución.

a) Creamos un *lock* por cada elemento del array, quedando

```

1 class Contador {
2     private int[] valor = new int[n]
3     private lock[] locks = new lock[n]
4     private int tamano
5     Contador(int n) {
6         tamano = n;
7         for (int i = 0; i < n; i++) {
8             valor[i] = 0
9         }
10    }
11    public void inc(int id) {
12        locks[id].lock()
13        valor[id % tamano]++
14        locks[id].unlock()
15    }
16    public int get() {
17        int total = 0;
18        for (int i = 0; i < tamano; i++) {
19            locks[i].lock
20        }
21        for (int i = 0; i < tamano; i++) {
22            total = total + valor[i];
23        }
24        for (int i = 0; i < tamano; i++) {
25            locks[i].lock
26        }
27        return total;
28    }
29 }
```

Notar que en `get()` primero tomamos todos los locks y luego hacemos las lecturas para calcular el acumulado. Esto es deliberado, estamos buscando garantizar el punto de linealización asegurándonos de que no pueda modificarse un contador mientras estamos leyendo otros.

La función `inc()` linealiza en el incremento, mientras que `get()` linealiza cuando se tomaron todos los locks. Dado que `inc()` toma sólo un lock, no llega a tener la posibilidad de estar en una espera circular. Múltiples llamados a `get()` podrían tener potencial de quedarse en espera circular, pero no lo harán porque toman todos los locks en el mismo orden.

b) Sólo modificamos `inc()`. Realizamos los incrementos por medio de una operación CAS en vez de utilizar un *lock*:

```

1 class Contador {
2
3     public void inc(int id) {
4         while (true) {
5             int valor_actual = valor[id%tamano]
```

```
6         CAS(valor[id%tamano],actual, actual + 1)
7     }
8 }
9 }
```

De esta manera, cada incremento es atómico (pero existe la posibilidad que de debamos reintentar si justo en el interín entre que leímos el valor actuar y quisimos realizar el **CAS** hubo un incremento).

- c) Es linealizable. La clave es notar que, si **get** logró hacer las dos lecturas y no detectó cambios, entonces podemos afirmar que los valores que leímos de las variables reflejan todos los incrementos realizados hasta algún punto en el tiempo (de no serlo, en la segunda lectura algún índice hubiera cambiado). Por lo tanto, podemos linealizar en cualquier punto después de la primera lectura completa del array que haya sido hecha en la **última iteración** de **get()** antes de terminar.
- d) No es *wait-free*, la terminación puede ser demorada (indefinidamente, incluso) si justo hay incrementos en el medio de los dos ciclos **for**, esto puede pasar una cantidad arbitraria de veces.

Inciso extra:

- Este es un caso especial donde la función en efecto es linealizable. El argumento es el siguiente: podemos observar que los incrementos son indistinguibles desde el punto de vista del contador total (el total es 3 tanto si **valor** es [1,2] como si es [3,0]). Es decir, si bien podemos tener una combinación de incrementos que no podría linealizar porque incluye incrementos en variables ya leídas por **get()**, lo cierto es que siempre podemos encontrar un punto en la línea de tiempo donde la cantidad de incrementos corresponde exactamente al valor observado en el acumulador. Luego, si tomamos a ese como el punto de linealización para **get()**, es consistente.

Por ejemplo, si hacemos **get()** y luego de haber leído **valor[0]** se hacen **inc(0)**, **inc(1)**, **\inc(0)**, **inc(1)**, la ejecución de **get()** va a leer los incrementos correspondientes a los **inc(1)** pero no los correspondientes a **inc(0)**, es decir, su lectura del estado de **valor** sería [0,2], lo cual no se correspondería con ningún punto en el tiempo. Sin embargo, como [0,2] para el contador es equivalente a [1,1] (porque los incrementos son indistinguibles), por lo que podemos en efecto poner el punto de linealización después de los dos primeros **incs**.